LAPORAN PRAKTIKUM

STRUKTUR DATA

“ABSTRACT DATA TYPE”

disusun Oleh:

NABIL FIKRI

2511533011

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

SHERLY SUKMADIRA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum mata kuliah **Struktur Data** ini tepat pada waktunya.

Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum mengenai implementasi kelas dan objek melalui studi kasus pembuatan ADT (Abstract Data Type) Jam menggunakan bahasa pemrograman Java. Penulis menyadari bahwa keberhasilan penyusunan laporan ini tidak terlepas dari bantuan berbagai pihak. Oleh karena itu, penulis ingin menyampaikan terima kasih kepada:

1. Dr. Wahyudi S.T,M.T, selaku dosen mata kuliah yang telah memberikan bimbingan teori di kelas.
2. Kakak asisten praktikum yang telah memberikan arahan dan bantuan teknis selama proses praktikum berlangsung di laboratorium.
3. Rekan-rekan mahasiswa angkatan 2025 Informatika Universitas Andalas yang telah saling mendukung dalam proses belajar.

Penulis menyadari bahwa laporan ini masih jauh dari kesempurnaan, baik dari segi materi maupun sistematika penulisannya. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang. Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan perkembangan ilmu pengetahuan, khususnya di bidang Informatika.

Padang, 01 April 2026

Nabil Fikri

DAFTAR ISI

KATA PENGANTAR i

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan Praktikum 1

1.3 Manfaat Praktikum 1

BAB II ISI 2

2.1 Landasan Teori 2

2.1.1 Atribut (Instance Variables) 3

2.1.2 Konstruktor (Constructor) 3

2.1.3 Selektor (Getter/Accessor) 3

2.1.4 Mutator (Setter) 3

2.1.5 Metode Validasi (Validator) 4

2.1.6 Overriding Method (toString) 4

2.2 Penjelasan Kode Program 5

2.2.1 Kelas Utama: Jam_2511533011.java 5

2.2.2. Kelas Driver Interaktif: JamDriver2_2511533011.java 6

BAB III PENUTUP 7

3.1 Kesimpulan 7

3.2 Saran 7

DAFTAR PUSTAKA 7

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman berorientasi objek (Java), representasi entitas dunia nyata ke dalam bentuk objek adalah hal yang fundamental. Salah satu entitas yang sering digunakan adalah waktu atau "Jam". Pembuatan kelas Jam_2511533011 ini bertujuan untuk memahami bagaimana data (jam, menit, detik) dapat dibungkus dalam sebuah kelas (*encapsulation*) dan bagaimana melakukan manipulasi aritmatika serta relasional terhadap data tersebut. Selain itu, penggunaan *static method* untuk validasi dan konversi menjadi poin penting dalam efisiensi kode.

1.2 Tujuan Praktikum

1. Mampu mengimplementasikan konsep kelas dan objek dalam bahasa pemrograman Java.

2. Memahami penggunaan akses kontrol *private* dan penyediaan akses melalui *getter* dan *setter*.

3. Menerapkan logika aritmatika waktu, seperti konversi ke detik dan penanganan *overflow* waktu (24 jam).

4. Mampu melakukan interaksi antara *user* dan program melalui kelas *driver* dan `Scanner`.

1.3 Manfaat Praktikum

1. Praktikan memahami cara membuat tipe data bentukan (ADT - *Abstract Data Type*) yang solid dan tervalidasi.
2. Meningkatkan kemampuan dalam menangani logika komputasi waktu yang sering ditemui dalam pengembangan aplikasi.
3. Melatih kemampuan dokumentasi kode yang bersih sesuai standar penamaan Java.

BAB II
ISI

2.1 Landasan Teori

Berdasarkan referensi dari *Deitel & Deitel (2017)* dan dokumentasi resmi *Oracle Java*, terdapat beberapa pilar utama yang diterapkan dalam praktikum ini:

- **Encapsulation (Penyembunyian Data):** Menurut *Schildt (2018)*, enkapsulasi adalah mekanisme yang mengikat kode dan data yang dimanipulasinya, serta menjaganya tetap aman dari interferensi luar. Dalam kode ini, penggunaan modifier **private** pada variabel **hh**, **mm**, dan **ss** memastikan integritas data jam.
- **Abstract Data Type (ADT):** Jam diperlakukan sebagai ADT, di mana pengguna kelas tidak perlu tahu bagaimana data disimpan secara internal, melainkan hanya perlu tahu operasi apa saja yang tersedia (seperti **plus**, **minus**, dan **compareTo**).
- **Method Overriding:** Praktikum ini menerapkan *overriding* pada metode **toString()** dan **equals()**. *Overriding* adalah kemampuan sub-kelas (atau kelas yang mewarisi **Object**) untuk memberikan implementasi spesifik dari metode yang sudah didefinisikan oleh super-kelasnya.
- **Static vs Instance Members:** Metode **static** (seperti **isValid**) dapat dipanggil tanpa instansi objek karena bersifat milik kelas, sedangkan metode *instance* (seperti **toSeconds**) bekerja pada data spesifik milik objek tersebut.

Dalam pemrograman berorientasi objek menggunakan Java, sebuah kelas (Class) terdiri dari beberapa komponen standar yang mengatur bagaimana data disimpan, diakses, dan dimodifikasi. Berikut adalah penjelasan mendalam mengenai komponen tersebut:

2.1.1 Atribut (Instance Variables)

Atribut adalah variabel yang dideklarasikan di dalam kelas tetapi di luar metode. Atribut merepresentasikan "state" atau keadaan dari sebuah objek.

- **Dalam Kode:** private int hh, mm, ss;.
- **Teori:** Penggunaan modifier private adalah penerapan **Data Hiding**. Variabel ini hanya bisa diakses oleh metode di dalam kelas yang sama, sehingga melindunginya dari perubahan yang tidak disengaja oleh kelas lain.

2.1.2 Konstruktor (Constructor)

Konstruktor adalah metode khusus yang dipanggil saat sebuah objek pertama kali dibuat (*instansiasi*) menggunakan kata kunci new.

- **Dalam Kode:** public Jam_2511533011(int h, int m, int s).
- **Teori:** Konstruktor tidak memiliki tipe pengembalian (*return type*) dan namanya harus sama persis dengan nama kelas. Fungsinya adalah memberikan nilai awal (inisialisasi) pada atribut objek.

2.1.3 Selektor (Getter/Accessor)

Selektor adalah metode yang digunakan untuk mengambil atau membaca nilai dari atribut yang bersifat *private*.

- **Dalam Kode:** public int getHour(), getMinute(), dan getSecond().
- **Teori:** Metode ini biasanya diawali dengan kata get. Tujuannya adalah memberikan akses baca kepada pengguna kelas tanpa memberikan izin untuk mengubah nilai aslinya secara langsung.

2.1.4 Mutator (Setter)

Mutator adalah metode yang digunakan untuk mengubah atau memberikan nilai baru pada atribut *private*.

- **Dalam Kode:** public void setHour(int h), setMinute(int m), dan setSecond(int s).
- **Teori:** Metode ini diawali dengan kata set. Mutator sangat penting karena kita bisa menyisipkan logika validasi di dalamnya (misalnya memastikan jam tidak lebih dari 23) sebelum nilai benar-benar disimpan ke atribut.

2.1.5 Metode Validasi (Validator)

Metode ini berfungsi untuk memastikan data yang masuk sesuai dengan aturan bisnis (domain) dari objek tersebut.

- **Dalam Kode:** public static boolean isValid(int h, int m, int s).
- **Teori:** Dengan membuat metode ini menjadi static, kita bisa melakukan pengecekan data jam bahkan sebelum objek Jam itu sendiri dibuat.

2.1.6 Overriding Method (toString)

Metode toString() adalah metode bawaan dari kelas Object di Java.

- **Dalam Kode:** @Override public String toString().
- **Teori:** Secara default, toString() akan mengembalikan alamat memori objek. Dengan melakukan *overriding*, kita memaksa program untuk mengembalikan informasi yang lebih berguna bagi manusia, yaitu format jam HH:mm:ss.

2.2 Penjelasan Kode Program

Berikut adalah bedah logika dari tiga file program yang telah diimplementasikan:

2.2.1 Kelas Utama: Jam_2511533011.java

Kelas ini dirancang sebagai kelas **final**, yang berarti tidak dapat diturunkan kembali untuk menjamin keamanan logika waktu.

1. **Validator (isValid):** Menggunakan logika konjungsi (&&) untuk memastikan jam (0-23), menit (0-59), dan detik (0-59) berada dalam rentang valid.
2. **Konversi Detik (toSeconds & fromSeconds):** Merupakan inti dari efisiensi program. Dengan mengubah waktu ke total detik (*hh* x 3600 + mm x 60 + *ss*), operasi aritmatika menjadi jauh lebih sederhana daripada memanipulasi tiga variabel secara terpisah.
3. **Aritmatika Waktu:** * Metode **plus** dan **nextNSeconds** menangani penambahan waktu.
    - Metode **minus** dan **prevSecond** menggunakan **Math.floorMod** untuk menangani *underflow* (misalnya: 00:00:00 dikurangi 1 detik akan menghasilkan 23:59:59, bukan nilai negatif).
4. **Kontrak Objek (equals & hashCode):** Implementasi ini menjamin bahwa dua objek Jam yang memiliki waktu yang sama akan dianggap identik secara logis, bukan hanya secara referensi memori.

2.2.2 Kelas Driver: JamDriver_2511533011.java

File ini berfungsi sebagai unit testing sederhana. Program melakukan instansiasi dua objek Jam (a dan b) dengan nilai yang sudah ditentukan (*hardcoded*) untuk menguji fungsi:

- Penjumlahan dua objek Jam.

- Perhitungan durasi antar dua waktu dalam satuan detik.
- Perbandingan relasional menggunakan compareTo.

2.2.2. Kelas Driver Interaktif: JamDriver2_2511533011.java

File ini mengimplementasikan interaksi dengan pengguna menggunakan **java.util.Scanner.**

1. **Metode buatJamDariInput:** Menerapkan teknik *defensive programming* dengan *infinite loop* (**while(true)**). Program akan terus meminta input jika data yang dimasukkan tidak lolos validasi **isValid()**.
2. **Alur Eksekusi:** Program menerima dua input jam, lalu secara otomatis menampilkan seluruh hasil operasi mulai dari konversi detik, perbandingan posisi waktu (lebih awal/lambat), hingga hasil penjumlahan kedua waktu tersebut.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilaksanakan, dapat disimpulkan bahwa implementasi kelas Jam_2511533011 telah berhasil memenuhi kaidah pemrograman berorientasi objek yang baik. Penggunaan enkapsulasi melalui metode *getter* dan *setter* telah terbukti efektif dalam menjaga validitas data jam, menit, dan detik. Selain itu, metode konversi waktu ke dalam satuan detik terbukti menjadi solusi yang sangat efisien untuk menangani operasi aritmatika dan relasional antar objek waktu. Program ini juga telah berhasil menangani transisi waktu harian secara otomatis, sehingga perpindahan detik yang melewati batas 24 jam dapat terkelola dengan presisi tanpa menimbulkan kesalahan nilai. Secara keseluruhan, praktikum ini memberikan pemahaman mendalam mengenai pembuatan tipe data bentukan yang tangguh dan siap digunakan dalam skenario pengembangan perangkat lunak yang lebih kompleks.

3.2 Saran

Meskipun program ini sudah berjalan dengan baik, terdapat beberapa aspek yang dapat dikembangkan lebih lanjut untuk meningkatkan kualitasnya. Salah satu saran utama adalah penyempurnaan format pada metode `toString()` agar menyertakan nilai detik sehingga informasi waktu yang ditampilkan menjadi lebih lengkap dan akurat. Selain itu, disarankan untuk mengintegrasikan mekanisme penanganan eksepsi (*Exception Handling*) menggunakan blok `try-catch` pada kelas *driver* guna mengantisipasi kesalahan input tipe data dari pengguna yang dapat menyebabkan program terhenti secara paksa.

DAFTAR PUSTAKA

[1] P. Deitel and H. M. Deitel, "Java How to Program, Early Objects," Pearson Education, 2017. [Online]. [Accessed 1 April 2026].

[2] H. Schildt, "Java: The Complete Reference," McGraw-Hill Education, 2018. [Online]. [Accessed 1 April 2026].

LAPORAN TUGAS

1. TUGAS

Tugas Praktikum Pekan 1

- Buatlah sebuah program ADT mobil yang memiliki atribut: nama, tahun, cc, harga, dan merk
- Contoh: Avanza, 2020, 1300, 150000000, Toyota
- Buatlah method ADT mobil tersebut: tambah mobil, hapus mobil, Selektor dan mutator
- Buatlah 2 kelas: 1 kelas ADT mobil dan 1 kelas driver. Setiap kelas lengkapi dengan nim

Gambar 1

2. PROGRAM

2.1 Kelas Mobil

```java
package pekan1_2511533011;

public class Mobil_2511533011 {
        String nama;
        int tahun;
        int cc;
        double harga;
        String merk;

        // Constructor
        public Mobil_2511533011(String nama, int tahun, int cc, double harga, String merk) {
            this.nama = nama;
            this.tahun = tahun;
            this.cc = cc;
            this.harga = harga;
            this.merk = merk;
        }

        // Getter (Selektor)
        public String getNama() { return nama; }
        public int getTahun() { return tahun; }
        public int getCc() { return cc; }
        public double getHarga() { return harga; }
        public String getMerk() { return merk; }

        // Setter (Mutator)
        public void setNama(String nama) { this.nama = nama; }
        public void setTahun(int tahun) { this.tahun = tahun; }
        public void setCc(int cc) { this.cc = cc; }
        public void setHarga(double harga) { this.harga = harga; }
        public void setMerk(String merk) { this.merk = merk; }

        // Tampilkan data
        public void tampilkan() {
            System.out.println("Nama  : " + nama);
            System.out.println("Tahun : " + tahun);
            System.out.println("CC    : " + cc);
            System.out.println("Harga : " + harga);
            System.out.println("Merk  : " + merk);
            System.out.println("----------------------");
        }
```

Gambar 2.1

2.2 Kelas Driver

```java
package pekan1_2511533011;

public class Driver_2511533011 {
    public static void main(String[] args) {

        String nim = "2511533011";
        System.out.println("NIM: " + nim);

        // Array untuk menyimpan mobil
        Mobil_2511533011[] daftarMobil = new Mobil_2511533011[3];

        // Tambah mobil
        daftarMobil[0] = new Mobil_2511533011("CRV", 2014, 2000, 100000000, "Honda");
        daftarMobil[1] = new Mobil_2511533011("X-TRAIL", 2016, 2000, 180000000, "Nissan");
        daftarMobil[2] = new Mobil_2511533011("BYD Seal", 2023, 1500, 200000000, "BYD");

        System.out.println("\nData Mobil:");
        for (int i = 0; i < daftarMobil.length; i++) {
            if (daftarMobil[i] != null) {
                daftarMobil[i].tampilkan();
            }
        }

        // Hapus mobil
        System.out.println("\nMenghapus mobil index 0 :");
        daftarMobil[0] = null;

        System.out.println("Setelah dihapus:");
        for (int i = 0; i < daftarMobil.length; i++) {
            if (daftarMobil[i] != null) {
                daftarMobil[i].tampilkan();
            } else {
                System.out.println("Data kosong di index " + i);
            }
        }

        //  Mutator
        System.out.println("\nUpdate harga mobil index 1 :");
        if (daftarMobil[1] != null) {
            daftarMobil[1].setHarga(200000000);
        }
```

Gambar 2.2

2.3 Output Program

```
NIM: 2511533011

Data Mobil:
Nama  : CRV
Tahun : 2014
CC    : 2000
Harga : 1.0E8
Merk  : Honda
----------------------
Nama  : X-TRAIL
Tahun : 2016
CC    : 2000
Harga : 1.8E8
Merk  : Nissan
----------------------
Nama  : BYD Seal
Tahun : 2023
CC    : 1500
Harga : 2.0E8
Merk  : BYD
----------------------

Menghapus mobil index 0 :
Setelah dihapus:
Data kosong di index 0
Nama  : X-TRAIL
Tahun : 2016
CC    : 2000
Harga : 1.8E8
Merk  : Nissan
----------------------
Nama  : BYD Seal
Tahun : 2023
CC    : 1500
Harga : 2.0E8
Merk  : BYD
----------------------

Update harga mobil index 1 :
Setelah update:
Nama  : X-TRAIL
```

```
Tahun : 2016
CC    : 2000
Harga : 2.0E8
Merk  : Nissan
----------------------
Nama  : BYD Seal
Tahun : 2023
CC    : 1500
Harga : 2.0E8
Merk  : BYD
----------------------
Merk mobil index 1: Nissan
```

2.4 Penjelasan

Kode program terdiri dari dua bagian utama, yaitu kelas entitas dan kelas *driver*. Kelas Mobil_2511533011 mendefinisikan lima atribut utama: nama, tahun, cc, harga, dan merk. Kelas ini dilengkapi dengan *Constructor* untuk inisialisasi, serta metode tampilkan() untuk memformat *output* data mobil agar mudah dibaca. Di sisi lain, Driver_2511533011 berperan sebagai pusat kendali program yang melakukan instansiasi array berkapasitas tiga elemen. Program mendemonstrasikan siklus hidup data (CRUD sederhana), dimulai dari pengisian data mobil (CRV, X-TRAIL, BYD Seal), penghapusan data pada indeks ke-0 dengan mengubahnya menjadi null, hingga proses pembaruan harga menggunakan metode setHarga. Penggunaan logika percabangan if (daftarMobil[i] != null) pada setiap perulangan sangat krusial untuk mencegah terjadinya NullPointerException saat mencoba mengakses data pada indeks yang telah dihapus.